# Que quepa
# y que corra rápido.

Su sistema ya hace todo lo que debe. Ahora pesa. Hoy abrimos el Módulo 5 con la primera palanca de eficiencia: destilación, un modelo chico que aprende del grande. Y la medimos con el harness de siempre.

DESTILACIÓN

TRADE-OFF

M5

# La sesión de hoy.

- Recap: resolvemos las dudas del formulario de salida
- Por qué eficiencia: latencia, costo y dónde corre su sistema
- Destilación: profesor y estudiante, etiquetas suaves, temperatura
- Tres formas de destilar: logits · datos sintéticos · el sistema entero — Lab
- Casos clásicos: DistilBERT, TinyLlama, Phi — y lo que viene en S15 (cuantizar y desplegar)
- Cierre: qué llevan a S15 y la entrega final (demo day 5 NOV)

# Primero,
# sus dudas.

Cuatro temas del formulario: elegir el patrón y qué pasa con dos tools parecidas · el juez VLM: cómo funciona y cómo saber si es confiable · la ética en la industria · cómo se despliega un modelo y si el flujo de SD es el estándar.

# Elegir el patrón, y qué pasa cuando dos tools se parecen

*"¿cómo elijo entre input, tool y output?" · "¿y si hay dos tools visuales con descripciones parecidas, cómo elige el agente?"*

## 1 · ELEGIR EL PATRÓN: una sola pregunta, tres respuestas

**¿De dónde viene la imagen?**

*no "qué modelo", no "qué es más vistoso": de dónde viene*

### La trae el USUARIO

*con su pregunta, siempre*

**INPUT**

un VLM la lee → texto

*ej.: "¿qué le pasa a esta hoja?" + foto*

### La busca el AGENTE

*solo a veces, cuando hace falta*

**TOOL**

CLIP / ViT clasifican → clase

*ej.: "¿llegó dañado?" → tool clasificar()*

### No existe todavía

*la respuesta es mejor dibujada*

**OUTPUT**

difusión → imagen marcada

*ej.: "¿cómo se vería el sofá en mi sala?"*

**Si dudan entre dos:**

- elijan el que responde la pregunta de su usuario con MENOS piezas nuevas
- input vs. tool: ¿la foto llega SIEMPRE con la pregunta? → input. ¿solo a veces? → tool
- output solo si dibujar responde mejor que escribir. Y "ninguno, medido" también vale

## 2 · DOS TOOLS PARECIDAS: el agente elige por el TEXTO de la descripción

*el LLM no ve el código de la tool; solo lee su nombre y su descripción. Si dos se parecen, elige al azar.*

### ASÍ NO: descripciones gemelas

```
analizar_imagen: "analiza una imagen"
ver_foto: "procesa una foto"
```

El agente no tiene cómo distinguirlas: a veces llama una, a veces la otra, a veces las dos. Y el harness lo muestra: "¿llamó la tool correcta?" sale 50 %.

### ASÍ SÍ: cuándo sí y cuándo no

```
clasificar_hoja: "SOLO si hay foto de una
hoja. Devuelve roya|minador|sana."
```

Cada descripción dice: qué entra, qué sale, CUÁNDO usarla y cuándo NO. Son excluyentes entre sí. Y si dos hacen casi lo mismo: una sola tool con un parámetro (modo=...).

**Tres reglas, y una forma de medirlo:**

1. **Excluyentes**
   cada tool tiene un "úsala cuando..." y un "NO la uses cuando..." que no se cruzan con las otras.
2. **Una tool, un trabajo**
   si dos tools comparten el 80 % de la descripción, son una sola con un parámetro.
3. **Ejemplos en la descripción**
   "ej.: el usuario adjunta foto de una hoja y pregunta por manchas" — el LLM aprende del ejemplo más que de la regla.

**Se mide (S10):** casos donde debe llamar la tool A, casos de la B, casos de ninguna → ¿eligió bien? Es una fila más del scorecard.

**El patrón lo decide de dónde viene la imagen. La tool la decide su descripción: excluyente, con ejemplos, y medida.**

*la descripción de una tool es un hiperparámetro del sistema: se escribe, se prueba y se compara como cualquier prompt*

# El juez VLM: cómo funciona, y cómo saber si es confiable

*"el juez VLM puede alucinar igual que el sistema que evalúa: ¿cómo sabemos que el juez es confiable?" — midiendo al juez*

## 1 · CÓMO FUNCIONA: el juez no califica "qué tan bueno"; responde una pregunta cerrada mirando la imagen



*la imagen*
*(la misma del caso)*

**lo que dijo SU sistema**

*"Es un cachorro beagle con una lesión en la pata izquierda."*

*← la afirmación a verificar*

**JUEZ VLM (Qwen2-VL, GPT-4o…)**

```
"¿Se ve una lesión en la pata
izquierda? Responde SOLO SÍ o NO,
y en qué parte de la imagen."
```

**NO**

*"no hay lesión visible; las patas están sanas"*

**nota del caso: 0**

**Fíjense en tres cosas:**

- pregunta CERRADA (sí/no), no "califica de 1 a 10"
- una afirmación a la vez, no la respuesta entera
- pide evidencia: "¿en qué parte?" — si no la da, no cuenta

*Es RAGAS (S10) con la imagen como contexto: ¿se sostiene?*

## 2 · ¿Y SI EL JUEZ ALUCINA? Se mide al juez igual que al sistema: contra humanos, en una muestra pequeña

**EL EXPERIMENTO (una hora de trabajo)**

1. **Tomen 20–30 casos con imagen y califíquenlos USTEDES**
   a mano: sí/no por afirmación. Eso es el "gold" del juez.
2. **Pásenle los mismos casos al juez VLM**
   con la misma pregunta cerrada.
3. **Cuenten en cuántos coincide**
   acuerdo juez–humano. Y miren DÓNDE falla: ¿dice SÍ a cosas que no están (alucina) o NO a cosas que sí están?
4. **Decidan con ese número**
   ≥ 85–90 % de acuerdo: úsenlo. Menos: cambien la pregunta, el modelo, o califiquen a mano.

**Lo que sale: una tablita de 2 × 2**

| | humano: SÍ | humano: NO |
|---|---|---|
| juez: SÍ | 11 | 2 ← alucinó |
| juez: NO | 1 ← no vio | 16 |

*acuerdo = 27 / 30 = 90 %. Los 2 "SÍ" falsos son alucinaciones: léanlos.*

**Para que el juez alucine MENOS:**

- preguntas cerradas y concretas, una por afirmación
- "describe primero lo que ves, luego responde" (S11)
- pide la evidencia: dónde en la imagen; sin evidencia = NO
- juez ≠ sistema: otro modelo, o al menos otro prompt
- en lo crítico (salud, legal): el juez propone, un humano decide
- promedio de 3 pasadas (o 3 jueces) en los casos dudosos

**La idea que resuelve la duda:**

No necesitan un juez perfecto. Necesitan un juez MEDIDO: saben en qué porcentaje coincide con ustedes y en qué tipo de caso se equivoca.

Un juez con 90 % de acuerdo que califica 200 casos en un minuto vale más que un humano que califica 20 y se cansa. Y los 3 casos dudosos los revisan a mano.

*Es la misma lógica de M2: la vara también se calibra. "Validamos el juez contra 30 etiquetas humanas: 90 % de acuerdo" es una frase para la demo.*

**Pregunta cerrada + evidencia + juez medido contra humanos. El juez no es confiable "porque sí": es confiable porque lo midieron.**

*en el notebook de S13, nota_juez_vlm ya hace la pregunta cerrada; hoy agregan la tablita 2 × 2*

# Ética en la industria: no es un párrafo al final, son controles en el sistema

*"¿cómo se manejan las consideraciones éticas en la industria?" — con cosas que se pueden auditar; y ustedes ya hacen cinco de ellas*

### 1 Datos: consentimiento y anonimización

Qué datos entran, con qué permiso, cuánto tiempo se guardan. Caras, documentos y placas fuera. En Colombia: Ley 1581 de habeas data.

✓ **ustedes: fotos anonimizadas, dataset open source**

### 2 Model card: qué hace y qué NO

Cada modelo publicado lleva una ficha: para qué sirve, con qué datos se entrenó, dónde falla, para qué no debe usarse (Hugging Face las exige).

✓ **ustedes: la sección "límites" del README**

### 3 Humano en el circuito

En decisiones de alto impacto (salud, crédito, legal) el modelo propone y una persona decide. El sistema lo deja explícito en la interfaz.

✓ **ustedes: "esto parece" + válvula; el VLM no diagnostica**

### 4 Contenido sintético marcado

Imágenes y textos generados llevan marca visible y, cada vez más, marca de agua invisible (C2PA, SynthID). Ley europea de IA: obligatorio.

✓ **ustedes: la etiqueta "imagen generada"**

### 5 Red teaming: buscar cómo falla

Antes de salir, un equipo intenta romper el sistema: prompts adversariales, sesgos, fugas de datos. Lo que encuentra va al reporte.

✓ **ustedes: los casos adversariales del eval set**

### 6 Trazabilidad y monitoreo

Cada respuesta queda registrada (logging) con su contexto; se monitorea en producción si el comportamiento cambia. Si falla, se puede reconstruir por qué.

✓ **ustedes: logging + W&B en S15**

**¿Quién lo exige? Tres fuentes, de menos a más obligatorias:**

**Las plataformas**

Hugging Face pide model card y licencia; OpenAI/Anthropic/Google publican políticas de uso y filtros de seguridad en sus APIs.

**Los estándares**

NIST AI Risk Management Framework (EE. UU.), ISO/IEC 42001 (sistema de gestión de IA): listas de controles que las empresas adoptan y auditan.

**Las leyes**

Ley de IA de la Unión Europea (2024): clasifica por riesgo y obliga a marcar contenido sintético y a supervisión humana en alto riesgo. Colombia: habeas data (Ley 1581) y proyectos en curso.

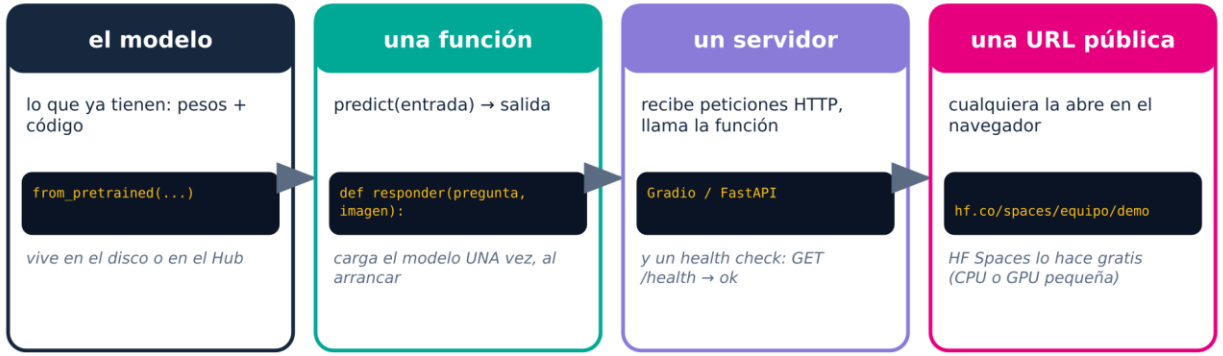
**En la industria la ética son controles auditables: datos, model card, humano en el circuito, marca, red team, trazabilidad.**

*cinco de los seis ya están en su proyecto; el sexto llega en S15. Eso es lo que va en la sección "limitaciones y ética" del reporte final*

# Dos preguntas rápidas: ¿cómo se despliega un modelo? · ¿el flujo de SD es el estándar?

*la primera es el tema de S15, pero la respuesta corta cabe en cuatro cajas; la segunda es un sí con dos reemplazos*

## 1 · DESPLEGAR = envolver el modelo en una función y ponerle una URL




**Lo que de verdad cambia frente al notebook:**

- el modelo se carga una vez y atiende muchas peticiones (en el notebook se carga cada vez que corren la celda)
- hay que medir latencia p50 / p95 por petición y decidir CPU vs. GPU: por eso primero destilamos (hoy) y cuantizamos (S15)
- alguien lo usa sin ustedes al lado: logging de cada petición y un health check que diga "estoy vivo"

**Las piezas, en orden de madurez (todas valen para la demo):**

Gradio en Colab con share=True → una URL temporal (72 h) · 5 líneas, hoy mismo
HF Spaces con Gradio → URL permanente, repo git, logs · lo que hacemos en S15
FastAPI + Docker en una nube (Azure, AWS, GCP) → lo que verían en una empresa; fuera del alcance del curso

## 2 · ¿prompt → CLIP → U-Net → VAE es EL estándar?

*Sí: la receta de todos los modelos abiertos desde 2022. Cambian dos piezas, no la receta.*

| | texto | quita ruido | comprime |
|---|---|---|---|
| **SD 1.5 · 2022 (el del lab)** | CLIP | U-Net | VAE 8× |
| **SDXL · 2023** | CLIP ×2 | U-Net (grande) | VAE 8× |
| **SD 3 · 2024** | CLIP ×2 + T5 | DiT (transformer) | VAE 8× |
| **Flux · 2024** | CLIP + T5 | DiT | VAE 8× |
| **DALL·E 3 · Midjourney** | cerrado | cerrado | cerrado |

**Las dos sustituciones que verán en los modelos nuevos:**

**U-Net → DiT:** el que quita ruido es un transformer sobre parches del latente (S12). Mismo trabajo, escala mejor.

**CLIP → CLIP + T5:** el traductor de texto se refuerza con un encoder de lenguaje grande: entiende prompts largos y texto en la imagen.

**Lo que NO cambia en ninguno:**

- el texto se codifica una vez y entra en cada paso
- el ruido se quita en un latente comprimido, N veces, con un scheduler
- un decoder revela la imagen al final
- guidance (CFG) y ControlNet/LoRA se enganchan igual

***Aprendieron la receta, no un ejemplo de clase.***

**Desplegar: modelo → función → servidor → URL (S15 lo hacemos). Y sí: lo de S12 es la receta estándar; solo cambian el traductor y el que limpia.**

*para la demo basta Gradio + HF Spaces; FastAPI y Docker son el siguiente paso fuera del curso*

# Su sistema pesa.

Un generador de 7 B, un retriever, un reranker, un VLM de 2 B y quizás un modelo de difusión. Funciona en Colab con una T4 prestada. ¿Y cuando lo use alguien más?

# Tres números que su usuario siente, aunque no los vea.

| COMPONENTE DE SU PIPELINE | PARÁMETROS | MEMORIA (fp16) | LATENCIA TÍPICA (T4) | QUIÉN LO NOTA |
|---|---|---|---|---|
| **generador Qwen2.5-7B (M1)** | 7 B | ~14 GB | 3–8 s por respuesta | el usuario, en cada mensaje |
| **retriever + reranker (M3)** | 0.1–0.5 B | < 1 GB | 0.3–1 s | casi nadie |
| **VLM Qwen2-VL-2B (M4)** | 2 B | ~4 GB | 2–5 s por imagen | el usuario, cuando manda foto |
| **Stable Diffusion 1.5 (M4 output)** | 1 B | ~2 GB | 4–6 s por imagen | el usuario, y la cuenta de GPU |
| **agente ReAct con 3 tools (M3)** | = generador × pasos | — | × 3–5 llamadas | todos: es lo que más tarda |

La pregunta de hoy no es "¿cómo lo hago mejor?" sino "¿cómo lo hago igual de bueno con menos?". Y "igual de bueno" se mide con el harness de M2: ese es el contrato.

Los números son órdenes de magnitud; los suyos salen del scorecard de M4 (latencia por caso) que trajeron hoy.

# Mismo resultado, menos cómputo: tres formas de lograrlo.

DESTILAR · HOY

## Un modelo chico aprende del grande

Entrenan un estudiante pequeño para imitar las salidas del profesor. Cambia el modelo: menos parámetros.

Gana: latencia y tamaño.

Paga: algo de calidad; horas de entrenamiento.

CUANTIZAR · S15

## Los mismos pesos, con menos bits

fp16 → int8 → int4. El modelo es el mismo; cada número ocupa 4× menos. Sin reentrenar.

Gana: memoria (4×) y algo de latencia.

Paga: muy poca calidad; nada de tiempo.

PEFT + SERVING · S15

## Adaptar barato y servir rápido

LoRA/QLoRA sobre un modelo ya cuantizado; vLLM o llama.cpp para atender muchas peticiones.

Gana: throughput.

Paga: configuración.

Se combinan: un modelo destilado Y cuantizado Y servido con vLLM es lo normal en producción. Hoy la primera; la próxima semana las otras dos, y el despliegue.

# Un estudiante que imita.

No aprende de las etiquetas: aprende de cómo responde (y cómo duda) un modelo más grande que ya sabe.

# Destilación: un modelo chico que aprende del grande, no de las etiquetas

*el profesor (teacher) ya sabe; el estudiante (student) no copia la respuesta: copia CÓMO duda el profesor*

## 1 · LA IDEA: las etiquetas duras tiran información; las "suaves" la conservan

**la pregunta**
*"¿esta hoja tiene roya?"*

**ETIQUETA DURA (del dataset)**

| | |
|---|---|
| roya | 1.0 |
| minador | 0.0 |
| sana | 0.0 |

*"es roya, punto" — un bit de información*

**ETIQUETA SUAVE (del profesor)**

| | |
|---|---|
| roya | 0.72 |
| minador | 0.23 |
| sana | 0.05 |

*"es roya, pero se parece al minador" — mucho más*

**PROFESOR · teacher**
grande, lento, bueno

Qwen2.5-7B · un ViT-L · su RAG completo
ya entrenado: NO se toca
solo se le pregunta, muchas veces

*860 M – 7 B parámetros · segundos por respuesta*

etiquetas suaves →

**ESTUDIANTE · student**
chico, rápido, aprende

Qwen2.5-0.5B · un ViT-tiny · regresión logística
se entrena a imitar las SALIDAS del profesor
sobre miles de entradas (etiquetadas o no)

*20 M – 500 M parámetros · milisegundos*

**Qué se minimiza (la única fórmula de hoy):**

```
pérdida = α·KL( softmax(z_prof/T) ‖ softmax(z_est/T) ) + (1−α)·CE(etiqueta, estudiante)
```

- KL: "qué tan distinta es mi distribución de la del profesor" — imitar la duda, no solo el acierto
- T (temperatura): al dividir los logits por T > 1 la distribución se "ablanda" y aparecen los parecidos (roya ≈ minador)
- α: cuánto pesa imitar al profesor vs. acertar la etiqueta. Si no hay etiquetas, α = 1 y funciona igual

## 2 · TRES FORMAS DE DESTILAR, de la más clásica a la que usan hoy

### A · Logits (Hinton, 2015)

*el estudiante imita las probabilidades del profesor, salida por salida*

DistilBERT: 40 % más chico, 60 % más rápido, 97 % del rendimiento. Para clasificadores y encoders (su CLIP, su ViT).

**en su proyecto:** clasificar hojas: ViT-base → ViT-tiny

### B · Datos sintéticos

*el profesor GENERA el dataset: preguntas y respuestas de su dominio; el estudiante se fine-tunea con eso (S04)*

Alpaca, Phi, Orca: modelos chicos entrenados con texto escrito por GPT-4. No hace falta ver los logits del profesor: sirve con una API cerrada.

**en su proyecto:** su generador: Qwen-7B escribe 2 000 pares → LoRA sobre Qwen-0.5B

### C · Destilar el SISTEMA

*el "profesor" es su pipeline completo (RAG + agente + tools); el estudiante aprende a dar la respuesta final sin el pipeline*

Lo que hacen las empresas cuando el agente es demasiado lento o caro para producción: el modelo chico memoriza el comportamiento del sistema grande.

**en su proyecto:** su RAG de M3 → un modelo chico que responde igual en el 80 % de los casos

**El trade-off, siempre medido con SU harness (M2):**

**CALIDAD**
¿cuánto baja el scorecard? (−2 puntos suele ser aceptable; −15 no)

**LATENCIA**
¿cuánto más rápido responde? (p50 y p95, en la misma máquina)

**TAMAÑO**
¿cabe donde antes no cabía? (CPU, celular, un Space gratis)

**Destilar = entrenar un modelo chico para imitar a uno grande. Se gana latencia y tamaño; se mide cuánta calidad se paga.**

*hoy en el lab: eligen UN componente pesado de su pipeline y lo destilan con la forma A o B*

# Con T = 1 el profesor casi no duda; con T = 4 muestra parecidos.

| CLASE | logit z | softmax(z / 1) | softmax(z / 4) | QUÉ APRENDE EL ESTUDIANTE |
|---|---|---|---|---|
| **roya** | 8.0 | 0.98 | 0.59 | es roya… |
| **minador** | 4.0 | 0.02 | 0.22 | …pero se parece al minador |
| **sana** | 1.0 | 0.00 | 0.10 | y poco a una hoja sana |
| **otra** | 0.0 | 0.00 | 0.09 | |

- **Con T = 1 la etiqueta suave es casi una etiqueta dura** — el estudiante solo aprende "roya" y pierde la información de parecido.
- **Con T > 1 (típico: 2–5) aparecen las relaciones entre clases** — "roya se confunde con minador, nunca con sana": eso es lo que vale la pena copiar del profesor.
- **Al evaluar, T vuelve a 1** — la temperatura es solo para entrenar; el estudiante en producción responde con su softmax normal.

Regla práctica: T = 2 para empezar; subirla si el estudiante aprende "demasiado duro".

# Lo que la industria ha destilado, y qué cedió a cambio.

| MODELO | PROFESOR → ESTUDIANTE | FORMA | QUÉ GANÓ | QUÉ CEDIÓ |
|---|---|---|---|---|
| **DistilBERT (2019)** | BERT-base 110 M → 66 M | A · logits + hidden states | 40 % menos tamaño, 60 % más rápido | ~3 % en GLUE (97 % del profesor) |
| **TinyBERT · MiniLM** | BERT → 15–30 M | A · atención y capas | cabe en un celular | 5–8 % en tareas difíciles |
| **Alpaca · Vicuna (2023)** | GPT-3.5/4 → LLaMA-7B | B · datos sintéticos | un chat útil con 52 k ejemplos | razonamiento largo, hechos raros |
| **Phi-3 · TinyLlama** | GPT-4 (datos) → 1–4 B | B · "textbooks" sintéticos | calidad de 7 B en 3.8 B | conocimiento del mundo |
| **SD-Turbo · SDXL-Turbo** | SD 50 pasos → 1–4 pasos | adversarial diffusion distillation | 10–25× más rápido (S12) | variedad y detalle fino |

Fíjense en la última columna: siempre se cede algo, y siempre está medido. Un modelo destilado sin la columna "qué cedió" no es ingeniería, es marketing.

# Una sola pieza. La que más tarda y menos calidad arriesga.

**1**

**MIREN SU SCORECARD DE M4: LATENCIA POR CASO**
La columna "segundos" dice dónde se va el tiempo. Casi siempre: el generador en el agente (× pasos) o el VLM.

**2**

**PREGUNTEN: ¿ESA PIEZA NECESITA SER GRANDE?**
Clasificar hojas o enrutar documentos: no (ruta A, un ViT-tiny o una logística). Redactar la respuesta final con citas: quizás sí (ruta B o C, con cuidado).

**3**

**ELIJAN LA FORMA**
A (logits) si es un clasificador o encoder suyo. B (datos sintéticos + LoRA) si es el generador. C (el sistema) solo si el agente entero es el cuello de botella.

**4**

**DEFINAN EL CONTRATO ANTES DE ENTRENAR**
"Acepto perder hasta X puntos del scorecard si gano Y segundos". Si no lo escriben antes, después cualquier resultado parece bueno.

Para la entrega final basta UNA técnica de eficiencia sobre UN componente, con la tabla antes/después. No destilen todo: destilen lo que importa.

# Al notebook · Destilar un componente

## Profesor, estudiante, y la tabla antes/después

Parte A (todos): destilan un clasificador por logits — un ViT-base (profesor) enseña a un modelo chico (estudiante) sobre imágenes de su dominio, con etiquetas suaves y temperatura; comparan accuracy, latencia y tamaño. Parte B (si su cuello de botella es el generador): el profesor escribe pares pregunta-respuesta de su dominio y fine-tunean con LoRA un Qwen2.5-0.5B. Cierran con la tabla del trade-off en su harness.

```
profesor → logits suaves (T) → entrenar estudiante → harness: calidad · latencia · tamaño → tabla antes/después   ·   S14_Lab_Destilacion.ipynb
```

Clase grabada: pausen el video aquí y trabajen el lab con su equipo (~45 min). Vuelvan para el cierre.

# Lo que tiene que existir ahora.

- **Un estudiante que corre** — más chico y más rápido que el profesor, sobre las mismas entradas. Con sus parámetros y su memoria anotados.
- **La tabla antes/después** — calidad (su harness o accuracy), latencia p50/p95 y tamaño, para profesor y estudiante en la misma máquina.
- **El contrato, cumplido o no** — "aceptábamos perder hasta X": ¿se cumplió? Si no, ¿subirían T, más datos, un estudiante más grande?
- **El caso donde el estudiante falla y el profesor no** — léanlo: ahí está lo que la destilación no pudo copiar.

Esa tabla es la mitad de M5. La otra mitad (cuantizar, desplegar, observar) es la próxima semana.

# Más chico
# no es gratis.

Un estudiante copia los aciertos del profesor… y también sus sesgos, con menos capacidad para matizarlos. Y hay cosas que no se pueden destilar.

# Tres límites y una regla de licencias.

### Los sesgos se heredan, y se amplifican

El estudiante aprende la distribución del profesor, incluidos sus errores sistemáticos. Con menos parámetros, pierde los matices que a veces los corregían. Midan al menos un caso de sesgo antes y después.

### Lo raro se pierde primero

La destilación conserva lo frecuente. Clases raras, preguntas poco comunes, el vocabulario de su dominio: ahí cae más la calidad. Su eval set debe tener casos raros para verlo.

### La válvula de escape puede desaparecer

Un profesor que dice "no se aprecia en la imagen" en el 5 % de los casos puede producir un estudiante que nunca lo dice. Los casos adversariales del eval set lo detectan.

### Las licencias: ¿se puede usar ese profesor para entrenar?

OpenAI, Anthropic y Google prohíben usar sus salidas para entrenar modelos competidores. Llama, Qwen, Mistral lo permiten con condiciones. Léanla antes; va en el reporte.

Para el reporte final: la sección "limitaciones y ética" incluye qué cedió el modelo destilado y qué licencia tiene el profesor.

# La próxima:
# cuantizar y desplegar.

Con el estudiante en la mano: menos bits, una URL pública, logging y un health check. Es lo que la demo necesita.

# De un modelo en Colab a un sistema con URL.

**Cuantización**
int8 e int4 con bitsandbytes y GGUF: el mismo modelo en 4× menos memoria. Demo en vivo: Qwen-7B en fp16 vs. int4, misma pregunta, mismo harness.

**Serving**
vLLM y llama.cpp: atender muchas peticiones a la vez. Latencia p50 y p95: la segunda es la que su usuario recuerda.

**Despliegue en HF Spaces**
Gradio + requirements + un Space: URL pública, repo git, logs. El lab de S15 es ese despliegue, instrumentado.

**Observabilidad con W&B**
Trazas de cada petición, métricas de uso, health check. "Producción verificable" en la rúbrica de la demo vale 4 de 30 puntos.

Traigan a S15: el estudiante de hoy (o el componente que eligieron), su tabla antes/después, y una cuenta de Hugging Face con un Space creado (vacío está bien).

# Demo day · jue 5 NOV · 15 min por equipo · 30 % · nota individual.

M5 · EN EL REPO

## Lo que va junto a M1–M4

Sistema desplegado con URL pública (HF Spaces).

UNA técnica de eficiencia medida: la tabla antes/después de hoy (destilar) o de S15 (cuantizar).

Health check, logging estructurado, observabilidad en W&B.

Reporte final + README: decisiones, scorecard vs. baselines, ablation, errores, límites y ética.

LA PONENCIA

## Los 15 minutos

Problema y usuario (1:30) · el sistema en un dibujo (1:30) · tres casos en vivo: fácil, difícil, adversarial (5:00) · los números contra baselines (3:00) · lo que falla y los límites (2:00) · preguntas (2:00).

Cada número con su baseline. Cada integrante puede explicar cualquier módulo.

La infografía completa está en Teams.

Hoy ganaron la fila "latencia antes/después" de su tabla de números. Es una de las frases de la demo: "bajamos de 6.4 s a 1.9 s perdiendo 2 puntos del scorecard".

# Tres ideas de la misma familia.

### Speculative decoding

Un modelo chico "adivina" varios tokens y el grande solo verifica: 2–3× más rápido sin perder nada de calidad. El estudiante y el profesor trabajando juntos en producción.

### DPO y RLHF (alignment)

Cómo se le enseña a un modelo a preferir unas respuestas sobre otras: pares (mejor, peor) en vez de etiquetas. Es lo que convierte un modelo base en un asistente.

### Modelos pequeños hoy

Phi-3-mini (3.8 B), Gemma 2B, Qwen2.5-0.5B/1.5B: entrenados con datos destilados de modelos grandes. Para su demo, probablemente uno de estos es el generador correcto.

Fuera del alcance de M5. Pero si su demo necesita responder en menos de un segundo, speculative decoding es la conversación.

# Dos semanas para el demo day.

Hoy: la primera palanca de eficiencia, medida. La próxima semana: cuantizar, desplegar y observar. Y el 5 de noviembre: la demo. El sistema ya existe; lo que queda es que corra rápido, en una URL, y que lo cuenten bien.

**Traigan a S15**
El estudiante de hoy con su tabla antes/después · una cuenta en Hugging Face con un Space creado · su pipeline de M4 corriendo en Colab.

**Esta semana**
Lean la licencia del profesor que usaron. Escriban el contrato de eficiencia en el README ("aceptamos perder X por ganar Y"). Ensayen los 15 minutos una vez, con reloj.

Pregunta para llevarse: si su usuario solo pudiera esperar un segundo por respuesta, ¿qué pieza de su sistema sobreviviría?

# Más chico, igual de bueno.
# Y medido.

Un estudiante que imita al profesor; etiquetas suaves y temperatura para copiar la duda, no solo el acierto; tres formas de destilar; y una tabla que dice cuánto se ganó y cuánto se pagó. La próxima semana: cuantizar y desplegar. En dos: la demo.